

```
# sage rsa34.sage
Original message: 68
Encrypted: 20359478209673026666575729300021602944974979857161
4571350900065161850041697415899637471131654411078345941194934
1290888812610145921792559879965116520391218808283746822116184
4121723978704450399919898180612800054087393967487001608747738
8225702040266850463087240264978762564391061138934044502439495
6596520186629549549552695914532124678674470381039220599765908
1529842585659542824615354610556450493081379470665478798222696
0912984537761200644541239335566192841888315903440042076013946
6654316391180403866065650680444680839832765022153650230672434
8241262069638572050489086149010996404330287216508349053249354
29389062309087026
Decrypted: 68
```

Figure 6: The working of RSA

encoded before encryption. Line 6 encrypts the message and produces the ciphertext c using the public key (e, n) . Line 7 decrypts the ciphertext and retrieves the original message using the private key (d, n) . Finally, lines 8 to 10 print the original message, the encrypted message, and the decrypted message, respectively.

Upon execution of the program ‘rsa34.sage’, the output shown in Figure 6 is obtained. From the figure, it can be verified that decryption successfully retrieves the original message from the encrypted message.

Now, let us ask an important question: Why is the encrypted text so large, even when we are encrypting a small two-digit number like 68? Imagine placing a small object (our message) into an enormous safe (the modulus n). Even though the object is tiny, the safe must be huge to ensure security—and that is exactly how RSA is designed. RSA encryption transforms a message m into a ciphertext c using the formula $c = m^e \bmod n$, where e is the public exponent and $n = p \times q$ is the modulus, formed by multiplying two large prime numbers. In standard RSA-1024, the primes p and q are each 1024 bits long, making n approximately 2048 bits in size, or around 617 decimal digits. So, even if the message m is small, like 68, raising it to the power of e and reducing it modulo n naturally produces a ciphertext that spans the full range of n , potentially resulting in a very large encrypted value. This large ciphertext is not a bug; it is a necessary aspect of RSA’s design. The security of RSA relies on the computational difficulty of factoring a large semiprime, and to preserve this security, both encryption and decryption must operate within a very large number space.

This also highlights why RSA and other public-key cryptographic systems are not used to encrypt large volumes of data—they are computationally intense and

suited only for encrypting small messages, such as secret keys. In practical systems, RSA is typically used to securely exchange a relatively short symmetric key, which is then used for fast, large-scale encryption with algorithms like AES. The topic of secure key exchange will be discussed in detail later in this series.

Now, let us discuss the practical security of RSA, particularly with 1024-bit keys. The security of RSA relies on the computational

difficulty of factoring large semiprime numbers, that is, products of two large prime numbers. For RSA-1024, the most efficient known classical factoring algorithm is the General Number Field Sieve (GNFS). Even with massive computational resources—millions of CPU cores working in parallel—GNFS would take over 100 years to factor such a number. This makes RSA-1024 practically secure for most everyday applications, provided that the attacker is limited to classical computing methods.

The real threat to RSA lies in the potential of quantum computing. If a sufficiently large and fault-tolerant quantum computer were to be built, it could run Shor’s algorithm to factor the RSA modulus efficiently—possibly in just a few hours. However, as of 2025, this scenario remains entirely theoretical.

In summary, RSA-1024 remains practically secure against classical attacks, but it is no longer considered future-proof due to advances in quantum computing research. For high-security applications, RSA with 2048 or 3072 bits—or quantum-resistant algorithms—are increasingly being recommended to ensure long-term protection.

Now it is time to wrap up our discussion. In this article, we explored how SageMath is much more than just another Python dialect. We looked at several examples where SageMath outperforms standard Python, thanks in part to powerful libraries like FLINT and GMP. We concluded with a discussion and implementation of RSA encryption. In the next article, we will continue our journey into public-key cryptography and take a closer look at the underlying mathematical libraries. **END** 🐧

 By: Dr Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science.